

Titled Gallery Cards, Powered by a Database

A step-by-step guide using Supabase

Goal: turn the bare images on the Home and Pet gallery pages into clean cards (image + title), and load them from a database so the artist can post new pieces herself later. Written to be followed even if databases are new to you, with the technical detail you need to actually ship it.

How this guide is organized

- **Part 1** — Build the card look (HTML + CSS). You can do this today with the images you already have.
- **Part 2** — Move the card data into a Supabase database and load the galleries from it.
- **Part 3** — What it takes later to let the client post new art herself.

Why Supabase? It is a free, hosted database that comes with file storage for the images and a friendly web table-editor — which is exactly what you will hand the client later. It also lets the galleries stay as plain static pages; you only add a few lines of JavaScript.

Part 1 — The card look

A “card” is just the image wrapped in a container together with a title. Today each gallery item looks like this:

```
<section class="gallery">
  
  
  ...
</section>
```

The card version wraps each image and adds a title underneath:

```
<section class="gallery">
  <figure class="card">
    
    <figcaption class="card-title">The Chesley Home</figcaption>
  </figure>
  <!-- one .card per piece -->
</section>
```

Add this CSS to `styles.css`. It restyles the existing `.gallery` grid for cards and matches your sage/cream brand:

```

.gallery {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1.75rem;
  padding: 1.5rem;
}
.card {
  margin: 0;
  background: #ffffff;
  border-radius: 10px;
  overflow: hidden;
  box-shadow: 0 4px 14px rgba(58, 42, 31, 0.10);
  transition: transform .2s ease, box-shadow .2s ease;
}
.card:hover {
  transform: translateY(-4px);
  box-shadow: 0 10px 24px rgba(58, 42, 31, 0.18);
}
.card img {
  width: 100%;
  aspect-ratio: 4 / 3; /* keeps every card the same height */
  object-fit: cover;
  display: block;
}
.card-title {
  padding: 14px 16px;
  font-family: "Playfair Display", serif;
  font-size: 1.15rem;
  color: #2b2b2b;
  text-align: center;
}

/* makes the grid responsive on phones/tablets */
@media (max-width: 900px) {
  .gallery { grid-template-columns: repeat(2, 1fr); }
}
@media (max-width: 560px) {
  .gallery { grid-template-columns: 1fr; }
}

```

That alone gives you uniform, polished cards with real titles. Once it looks right, move on to Part 2 so you never hand-edit this HTML again.

Tip: notice the alt text now describes the image. Keep doing that — it helps screen readers and search engines.

Part 2 — Load the cards from a database

The plan: store one row per portrait in a Supabase table. Each row holds a title, which gallery it belongs to (home or pet), and the image's web address. The gallery pages ask Supabase for the matching rows and build the cards automatically.

Step 1 — Create a free Supabase project

- Go to supabase.com, sign up, and click **New project**. Give it a name (e.g. “slm-custom-art”) and a database password (save it somewhere).

- Wait a minute for it to provision. That's it — you now have a hosted database.

Step 2 — Create the table

In the Supabase dashboard open the **SQL Editor**, paste the following, and click Run. (You could click through the table UI instead, but SQL is faster and exact.)

```
create table portraits (  
  id          bigint generated always as identity primary key,  
  title       text    not null,  
  category    text    not null check (category in ('home','pet')),  
  image_url   text    not null,  
  display_order int    default 0,  
  created_at  timestamptz default now()  
);
```

Each column, in plain terms: **title** is what shows on the card; **category** tells the two pages apart; **image_url** is where the picture lives; **display_order** lets the client reorder pieces; **created_at** is filled in automatically.

Step 3 — Create a place to store the images

In **Storage**, create a new bucket named **portraits** and mark it **Public** (galleries need to read the images without a login). You can drag your existing photos from `/images/homes` and `/images/pets` into it. Each uploaded file gets a public URL like:

```
https://YOUR-PROJECT.supabase.co/storage/v1/object/public/portraits/chesley-home.jpg
```

That URL is what goes in the `image_url` column. (If you prefer, you can skip Storage at first and keep using your local `/images/...` paths in `image_url` — but Storage is what lets the client add images without touching the code, so it's worth setting up now.)

Step 4 — Add a few rows

Still in the SQL Editor, seed the table with your current pieces (swap in real titles and URLs):

```
insert into portraits (title, category, image_url, display_order) values  
( 'The Chesley Home', 'home',  
  'https://YOUR-PROJECT.supabase.co/storage/v1/object/public/portraits/chesley.jpg', 1),  
( 'Timber Lane',      'home',  
  'https://YOUR-PROJECT.supabase.co/storage/v1/object/public/portraits/timber.jpg', 2),  
( 'Lando',            'pet',  
  'https://YOUR-PROJECT.supabase.co/storage/v1/object/public/portraits/lando.jpg', 1);
```

Step 5 — Let the website read (but not change) the data

Supabase locks tables down by default with “Row Level Security.” You want the public to **read** portraits but not edit them. Run this once:

```
alter table portraits enable row level security;

create policy "Public can read portraits"
  on portraits for select
  to anon
  using (true);
```

This is the safe part to understand: you will put a Supabase key in your website's code, but it is the **public (anon) key**, and this policy only grants it permission to read. No one can insert or delete anything through it. Writing is reserved for the admin flow in Part 3.

Step 6 — Get your project keys

In **Project Settings** → **API**, copy two values: the **Project URL** and the **anon public** key. You'll paste them into the gallery pages next.

Step 7 — Load the cards on each gallery page

On `homes-gallery.html`, leave the gallery section empty — JavaScript will fill it — and add the Supabase client plus a small script before `</body>`:

```
<section class="gallery" id="gallery"></section>

<script src="https://cdn.jsdelivr.net/npm/@supabase/supabase-js@2"></script>
<script>
  const SUPABASE_URL = "https://YOUR-PROJECT.supabase.co";
  const SUPABASE_ANON_KEY = "your-anon-public-key";
  const db = supabase.createClient(SUPABASE_URL, SUPABASE_ANON_KEY);

  // change "home" to "pet" on the pets page -- that's the only difference
  const CATEGORY = "home";

  async function loadGallery() {
    const grid = document.getElementById("gallery");
    const { data, error } = await db
      .from("portraits")
      .select("*")
      .eq("category", CATEGORY)
      .order("display_order");

    if (error) { console.error(error); return; }

    grid.innerHTML = data.map(piece => `
      <figure class="card">
        
        <figcaption class="card-title">${piece.title}</figcaption>
      </figure>
    `).join("");
  }

  loadGallery();
</script>
```

For `pets-gallery.html`, use the exact same block but change one line to `const CATEGORY = "pet";`. Reload the pages — the cards now come straight from the database.

Tip: to keep your two pages tidy, you can move this script into a shared file (e.g. `gallery.js`) and read the category from a data-attribute on the section. Optional, but it avoids duplication.

Part 3 — Letting the client post new art (the road ahead)

Everything above already gives the client a no-code way to add pieces today: she can log into the Supabase dashboard, drag a photo into the Storage bucket, then add a row in the **Table Editor** with the title, category, and image URL. For many small businesses that is enough.

When you want a friendlier, branded “Add a portrait” page on the site itself, the pieces are:

- **A login** so only the artist can post — Supabase has built-in Auth (email + password) for this.
- **An upload form** that sends the photo to the Storage bucket and creates the matching row. Supabase's JavaScript library does both in a few lines.
- **A write policy** — a second RLS rule that allows `insert` for logged-in (authenticated) users only, while the public key from Part 2 stays read-only.

Because you designed the table and read policy correctly from the start, this is an additive step later — nothing built in Parts 1 and 2 has to be torn out.

Quick checklist

- Restyle `.gallery` and add `.card` CSS (Part 1).
- Create the Supabase project, the `portraits` table, and a public Storage bucket.
- Add the read-only RLS policy and seed a few rows.
- Drop the Supabase script into both gallery pages, setting `CATEGORY` to `home` / `pet`.
- Later: add Auth + an upload form + an insert policy for the client.